

ShopSphere – Project Architecture & Progress Tracker

1. Project Vision

ShopSphere is being built as a modern enterprise-grade E-Commerce backend platform using:

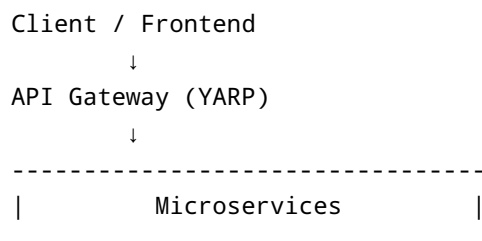
- .NET 9
- ASP.NET Core Web API
- Clean Architecture
- Microservices Architecture
- Docker
- Redis
- SQL Server
- API Gateway (YARP)
- JWT Authentication

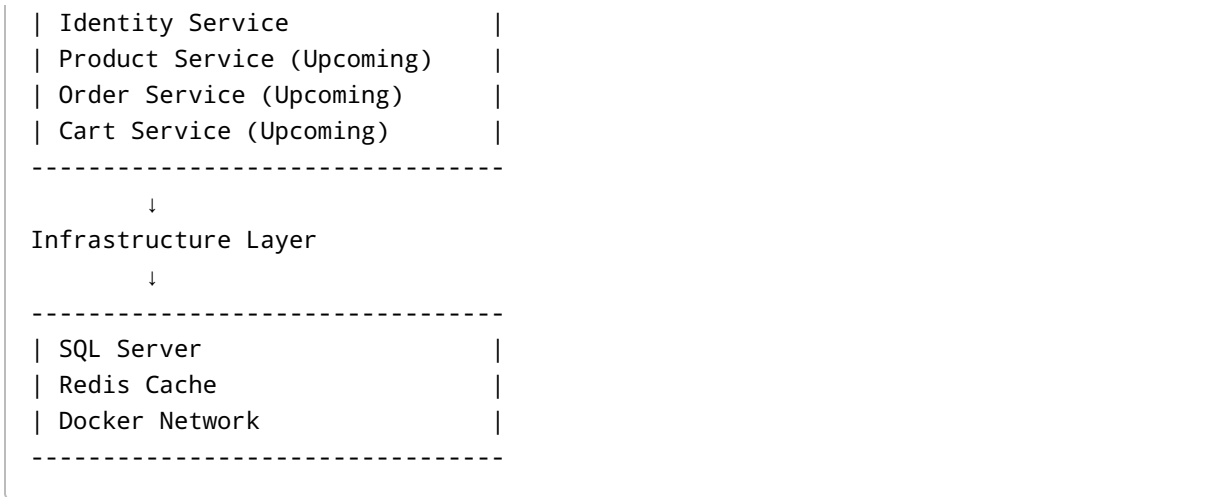
The goal of this project is not just CRUD development.

The goal is to learn and implement:

- Enterprise backend architecture
- Scalable distributed systems
- Production-ready authentication systems
- Containerized deployments
- API Gateway routing
- Caching
- Logging & observability
- Service-to-service communication
- CI/CD concepts
- Cloud-ready architecture

2. Current High-Level Architecture





3. Architecture Styles Used

3.1 Microservices Architecture

The project is divided into independent backend services.

Each service:

- owns its business logic
- owns its database access
- can be deployed independently
- communicates through APIs

Examples:

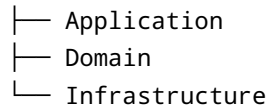
- Identity Service
- Product Service
- Order Service
- Cart Service

3.2 Clean Architecture

Inside each microservice we follow Clean Architecture.

Example:





Purpose of each layer:

API Layer

Contains:

- Controllers
- Middleware
- Swagger
- API configuration
- Dependency Injection setup

Responsibilities:

- Handle HTTP requests/responses
- Authentication/Authorization
- Middleware pipeline

Application Layer

Contains:

- DTOs
- Interfaces
- Business services
- Use cases
- Validation logic

Responsibilities:

- Application/business workflows
- Service orchestration
- Business rules coordination

Domain Layer

Contains:

- Entities
- Enums
- Domain models

Responsibilities:

- Core business rules
- Pure domain logic
- Independent of infrastructure

Infrastructure Layer

Contains:

- EF Core
- Repositories
- Redis
- JWT generation
- Logging setup
- External integrations

Responsibilities:

- Database access
- External system integrations
- Technical implementation details

4. Recommended Final Folder Structure

```
ShopSphere
|
|— services
|   |
|   |— IdentityService
|   |   |— ShopSphere.IdentityService.API
|   |   |— ShopSphere.IdentityService.Application
|   |   |— ShopSphere.IdentityService.Domain
|   |   └— ShopSphere.IdentityService.Infrastructure
|   |
|   |— ProductService
|   |   |— ShopSphere.ProductService.API
|   |   |— ShopSphere.ProductService.Application
|   |   |— ShopSphere.ProductService.Domain
|   |   └— ShopSphere.ProductService.Infrastructure
|   |
|   └— OrderService
```

```
|   └─ ShopSphere.ApiGateway
|
|   └─ docker
|       └─ docker-compose.yml
|
|   └─ frontend
|       ├── shopsphere-web
|       └─ shopsphere-admin
|
|   └─ docs
|       ├── architecture
|       ├── api-contracts
|       └─ setup-guides
|
|   └─ architecture
|       ├── diagrams
|       ├── decisions
|       └─ notes
|
|   └─ .github
|       └─ workflows
```

5. Purpose of Main Root Folders

services/

Contains all backend microservices.

Examples:

- Identity Service
- Product Service
- Order Service
- API Gateway

docker/

Contains:

- docker-compose.yml
- infrastructure setup
- local development orchestration

Used for:

- starting all containers together
 - local environment setup
 - container networking
-

frontend/

Contains frontend applications.

Examples:

- React web app
 - Admin dashboard
 - Mobile frontend later
-

docs/

Contains project documentation.

Examples:

- setup guides
 - API contracts
 - onboarding notes
 - architecture explanations
-

architecture/

Contains higher-level architecture design documents.

Examples:

- HLD diagrams
 - LLD diagrams
 - event-driven design notes
 - system decisions
-

.github/

Contains GitHub Actions workflows.

Used for:

- CI/CD
 - automated builds
 - testing pipelines
 - deployments
-

6. Infrastructure Overview

SQL Server

Purpose:

- persistent relational database
- stores users, products, orders, etc.

Current status:

- running locally on machine
- Docker containers connect using:

```
host.docker.internal,1433
```

Redis

Purpose:

- distributed caching
- fast lookup storage
- session/token optimization

Current status:

- running inside Docker container
 - profile caching implemented
-

Docker

Purpose:

- containerized deployments
- isolated environments
- production-like local setup

Current containers:

- shopsphere-api
 - shopsphere-gateway
 - shopsphere-redis
-

API Gateway (YARP)

Purpose:

- single entry point for clients
- reverse proxy routing
- centralized request handling

Current route:

```
http://localhost:7000/identity/swagger/index.html
```

Gateway forwards requests to:

```
shopsphere-api:8080
```

inside Docker network.

7. Features Completed So Far

Authentication System

Status:  Completed

Features:

- User Registration

- User Login
 - Password Hashing (BCrypt)
 - JWT Authentication
 - Claims-based tokens
 - Refresh Tokens
 - Token Revocation
 - Role support
-

Exception Middleware

Status:  Completed

Features:

- centralized error handling
 - consistent API responses
 - structured error format
-

Logging

Status:  Completed

Features:

- Serilog integration
 - file logging
 - request logging
 - authentication logs
-

Redis Caching

Status:  Completed

Features:

- profile caching
 - Redis integration
 - cache retrieval validation
-

Dockerization

Status: ☒ Completed

Features:

- API containerization
 - Redis containerization
 - docker-compose setup
 - container networking
-

API Gateway

Status: ☒ Completed

Features:

- YARP reverse proxy
 - route forwarding
 - gateway-based access
-

8. Progress Tracker

Phase 1 – Foundation

- ☒ Solution setup
 - ☒ Clean architecture setup
 - ☒ Identity Service setup
 - ☒ JWT authentication
 - ☒ Refresh tokens
 - ☒ Middleware
 - ☒ Logging
 - ☒ Redis caching
 - ☒ Docker setup
 - ☒ API Gateway
-

Phase 2 – Product Service

- ☐ Product Service setup
- ☐ Product CRUD
- ☐ Product caching

- ☐ Product validation
 - ☐ Swagger integration
 - ☐ Dockerization
 - ☐ Gateway integration
-

Phase 3 – Order Service

- ☐ Order creation
 - ☐ Order workflow
 - ☐ Inventory validation
 - ☐ Service-to-service communication
 - ☐ Distributed transactions concepts
-

Phase 4 – Advanced Backend Engineering

- ☐ RabbitMQ / Kafka
 - ☐ Distributed logging
 - ☐ Health checks
 - ☐ API versioning
 - ☐ Monitoring
 - ☐ Grafana
 - ☐ Prometheus
 - ☐ Elasticsearch
 - ☐ Kibana
-

Phase 5 – Cloud & DevOps

- ☐ Kubernetes
 - ☐ CI/CD pipelines
 - ☐ GitHub Actions
 - ☐ Azure/AWS deployment
 - ☐ Production environment setup
-

9. Important Learning Concepts Already Covered

Backend Concepts

- Dependency Injection
- Repository Pattern
- Middleware Pipeline

- Authentication & Authorization
 - DTO Pattern
 - Service Layer
 - Caching
 - Logging
 - Clean Architecture
 - Microservices
-

Infrastructure Concepts

- Docker
 - Docker Compose
 - Container Networking
 - Redis
 - Reverse Proxy
 - Gateway Routing
-

10. Current Running Endpoints

Identity Service Direct Access

```
http://localhost:5000/swagger/index.html
```

Gateway Access

```
http://localhost:7000/identity/swagger/index.html
```

11. Future Improvements

Potential future additions:

- Centralized authentication service
- Shared building blocks library
- Distributed tracing
- CQRS
- Event-driven communication
- Saga pattern

- Kubernetes deployment
 - API rate limiting
 - Monitoring dashboards
 - OpenTelemetry
 - Service discovery
-

12. Important Architectural Understanding

Microservice vs Clean Architecture

Microservice

Example:

```
IdentityService
```

A complete backend system.

Clean Architecture Layers

Inside IdentityService:

```
API  
Application  
Domain  
Infrastructure
```

These are implementation layers of ONE service.

Not separate services.

13. Current Status Summary

Current project maturity level:

Intermediate → Advanced Backend Engineering

Major accomplishments already completed:

- Enterprise architecture setup
- Authentication system
- Gateway routing
- Containerized backend
- Redis caching
- Logging
- Refresh tokens
- Docker networking

The project is now moving beyond tutorial-level CRUD development into production-style backend engineering.